

Lab 5 Distributed Parallel Computing Lab Programs

Question 1: Parallel Matrix Multiplication using OpenMP

Aim

To implement matrix multiplication using OpenMP parallel programming.

Problem Statement

Write a C program using OpenMP to perform multiplication of two matrices. The program should use parallel loops to improve performance and reduce execution time.

Algorithm

1. Start the program.
2. Define matrices A, B, and C.
3. Initialize matrices A and B with values.
4. Use OpenMP parallel directive to parallelize the outer loop.
5. Multiply matrices using nested loops.
6. Store the result in matrix C.
7. Print the result matrix.
8. End the program.

Program

```
1 #include <stdio.h>
2 #include <omp.h>
3
4 #define N 3
5
6 int main() {
7     int A[N][N] = {{1,2,3},{4,5,6},{7,8,9}};
8     int B[N][N] = {{1,0,0},{0,1,0},{0,0,1}};
9     int C[N][N];
10    int i,j,k;
11
```

```

12     #pragma omp parallel for private(j,k)
13     for(i = 0; i < N; i++) {
14         for(j = 0; j < N; j++) {
15             C[i][j] = 0;
16             for(k = 0; k < N; k++) {
17                 C[i][j] += A[i][k] * B[k][j];
18             }
19         }
20     }
21
22     printf("Result_Matrix:\n");
23
24     for(i = 0; i < N; i++) {
25         for(j = 0; j < N; j++) {
26             printf("%d_", C[i][j]);
27         }
28         printf("\n");
29     }
30
31     return 0;
32 }

```

Sample Output

Result Matrix:

```

1 2 3
4 5 6
7 8 9

```

Question 2: Parallel Bubble Sort using OpenMP

Aim

To sort an array using Bubble Sort with OpenMP parallelization.

Problem Statement

Write a C program using OpenMP to implement Bubble Sort. Use the Odd-Even Transposition method so that comparisons can run in parallel.

Algorithm

1. Start the program.
2. Read number of elements.
3. Input array elements.
4. Repeat for N iterations:
 - If iteration is even, compare elements (0,1), (2,3), (4,5).
 - If iteration is odd, compare elements (1,2), (3,4), (5,6).
5. Swap elements if they are out of order.
6. Print sorted array.
7. End program.

Program

```
1 #include <stdio.h>
2 #include <omp.h>
3
4 int main()
5 {
6     int n, i, j, temp;
7
8     printf("Enter number of elements: ");
9     scanf("%d", &n);
10
11     int arr[n];
12
13     printf("Enter array elements:\n");
14     for(i = 0; i < n; i++)
15         scanf("%d", &arr[i]);
16
17     for(i = 0; i < n; i++)
18     {
19         if(i % 2 == 0)
```

```

20     {
21         #pragma omp parallel for private(temp)
22         for(j = 0; j < n-1; j += 2)
23         {
24             if(arr[j] > arr[j+1])
25             {
26                 temp = arr[j];
27                 arr[j] = arr[j+1];
28                 arr[j+1] = temp;
29             }
30         }
31     }
32     else
33     {
34         #pragma omp parallel for private(temp)
35         for(j = 1; j < n-1; j += 2)
36         {
37             if(arr[j] > arr[j+1])
38             {
39                 temp = arr[j];
40                 arr[j] = arr[j+1];
41                 arr[j+1] = temp;
42             }
43         }
44     }
45 }
46
47 printf("Sorted array:\n");
48
49 for(i = 0; i < n; i++)
50     printf("%d ", arr[i]);
51
52 return 0;
53 }

```

Sample Output

Enter number of elements: 5

Enter array elements:

5 2 9 1 6

Sorted array:

1 2 5 6 9